Pegasus Spyware Detection

Amrutha Gowri Jayasimha Hanumesh
Course Project – Information Security
Professor: Izzat Alsmadi
Texas A&M University–San Antonio

***Abstract*—This comprehensive study presents a multi-phase machine learning approach to detecting Pegasus spyware through Indicators of Compromise (IoC) analysis. Spanning three progressive milestones, the research explores the complete machine learning pipeline from baseline establishment through advanced ensemble methods. Utilizing a synthetic Pegasus spyware dataset from Kaggle comprising 1,000 samples with 17 features across three target classes (None, Malicious IP, Pegasus Signature), this investigation systematically implements and evaluates multiple detection methodologies.**

Milestone 1 established baseline performance through comprehensive Exploratory Data Analysis (EDA), revealing critical behavioral patterns including $13.8\times$ elevated data volumes in Pegasus traffic and distinctive temporal clustering during night hours (33% activity concentration between 00:00–06:00). The baseline CatBoost classifier achieved 84–85% accuracy, validating dataset quality and establishing improvement targets.

Milestone 2 advanced detection capabilities through systematic feature engineering, comparing Principal Component Analysis (PCA), Chi-Square selection, and Mutual Information methods. Chi-Square feature selection emerged as the optimal approach, achieving 91.2–98% accuracy using only 15 carefully selected features—representing a 6–7% improvement over baseline. This phase included extensive literature review of 10 peer-reviewed papers and analysis of 5 Kaggle implementations, contextualizing the approach within existing mobile malware detection research.

Milestone 3 implemented comprehensive model evaluation across more than 17 algorithms spanning three methodological tiers: six classical machine learning models (Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbors, Support Vector Machine, Naive Bayes), three deep learning architectures (Multi-Layer Perceptron, 1D Convolutional Neural Network, LSTM), and five manually-constructed ensemble methods (Feature-based, Data-based, Model-based, Model-Instance, and Output-based). Critical technical challenges including severe data leakage from high-cardinality categorical features were systematically identified and resolved through strategic feature engineering.

The investigation achieved a maximum accuracy of 95% through a Soft Voting Ensemble, demonstrating that diversity-driven ensemble approaches outperform individual models. Comprehensive benchmarking revealed Random Forest (90.4% accuracy, 1.2s training) as optimal for production deployment due to superior balance of accuracy, speed, and generalization. Comparison with five Kaggle implementations validated the superiority of feature engineering over architectural complexity, with the developed models achieving competitive or superior performance despite smaller dataset sizes.

Key contributions include: (1) systematic validation of forensic indicators against Citizen Lab research, (2) demonstration that feature engineering (Milestone 2) provides greater accuracy gains than model complexity, (3) resolution of data leakage through principled feature exclusion, (4) comprehensive benchmark establishing ensemble method superiority, and (5) production-ready two-tier deployment strategy combining real-time Random Forest detection with batch-mode ensemble verification.

This research provides actionable insights for cybersecurity practitioners deploying machine learning-based intrusion detection systems, emphasizing that data quality and feature engineering fundamentally outweigh algorithmic sophistication in achieving production-viable spyware detection capabilities. The complete progression from 85% baseline to 95% ensemble accuracy represents a 10–11% total improvement, achieved through systematic application of sound machine learning principles rather than architectural complexity alone.

***Index Terms*—Pegasus spyware, intrusion detection, machine learning, deep learning, ensemble learning, feature engineering, indicators of compromise.**

I. Research Goal

A. Primary Objective

Develop and evaluate machine learning models capable of accurately detecting Pegasus spyware through analysis of system logs, network traffic patterns, and behavioral indicators of compromise (IoC).

B. Specific Research Questions

Milestone 1:

- What baseline detection accuracy can be achieved using standard classifiers with minimal feature engineering?
- What data quality characteristics and behavioral patterns distinguish Pegasus activity?

Milestone 2:

- Which feature engineering methodology (PCA, Chi-Square, Mutual Information) provides optimal balance of accuracy improvement and computational efficiency?
- How do our approaches compare to existing literature and Kaggle implementations?

Milestone 3:

- How do classical machine learning, deep learning, and ensemble methods compare in Pegasus detection?
- Can manual ensemble construction outperform individual base models?
- What deployment strategy optimizes real-world detection scenarios?

C. Success Metrics

- Achieve $> 80\%$ detection accuracy with minimal false positives.
- Demonstrate measurable improvement across milestone progression.

- Validate findings against established forensic research (Citizen Lab).
- Develop production-ready deployment recommendations.

II. Introduction

In the contemporary landscape of digital surveillance and cyber espionage, sophisticated spyware tools represent critical threats to individual privacy and organizational security. Pegasus spyware, developed by the NSO Group, exemplifies the apex of commercial surveillance technology, employing zero-click exploitation capabilities to compromise iOS and Android devices without user interaction. First publicly documented by Citizen Lab at the University of Toronto's Munk School, Pegasus has been deployed against journalists, human rights activists, politicians, and civil society members across multiple continents.

The technical sophistication of Pegasus presents unique detection challenges: traditional signature-based antivirus approaches fail against its polymorphic characteristics, while behavioral detection requires identification of subtle anomalies amidst legitimate system activity. The spyware employs advanced evasion techniques including encrypted communications, kernel-level access, and careful traffic mimicry designed to blend with normal device operations.

A. Research Significance

Machine learning approaches offer promising detection capabilities through pattern recognition across multiple behavioral dimensions. Unlike signature-based methods, ML models can identify statistical anomalies in network behavior, system calls, error patterns, and temporal activity that collectively indicate compromise. This research systematically explores ML-based detection across three progressive development phases, from baseline establishment through advanced ensemble methods.

B. Technical Context

Pegasus operates through multiple attack vectors including:

- Network-based exploitation of vulnerabilities in messaging protocols (WhatsApp, iMessage).
- Zero-click attacks requiring no user interaction for successful compromise.
- Privilege escalation achieving root/kernel-level access for complete device control.
- Data exfiltration: systematic extraction of messages, calls, location data, and stored files.
- Evasion: anti-forensic capabilities designed to resist detection and analysis.

Detection requires analysis across multiple indicators:

- Network patterns: volume anomalies, unusual protocols, encryption characteristics.
- Temporal behavior: activity clustering during unusual hours.
- System indicators: error codes, process signatures, file access patterns.
- Resource utilization: abnormal data volume, CPU usage, memory allocation.

C. Dataset Context

This investigation employs a synthetic Pegasus spyware detection dataset from Kaggle, comprising 1,000 samples with 17 features representing realistic forensic indicators. While synthetic, the dataset was constructed to reflect patterns documented in actual Pegasus forensic analyses, enabling controlled experimentation while respecting ethical constraints around sharing genuine malware artifacts.

D. Project Structure

The research progresses through three integrated milestones:

- **Milestone 1 (Baseline & EDA):** establishes foundational understanding through comprehensive exploratory analysis, validates data quality, implements a baseline classifier, and identifies key behavioral patterns distinguishing malicious activity.
- **Milestone 2 (Feature Engineering & Literature Review):** systematically compares dimensionality reduction and feature selection methodologies, contextualizes the approach within existing academic and practitioner literature, and achieves significant accuracy improvements through Chi-Square feature selection.
- **Milestone 3 (Comprehensive Model Evaluation):** implements 17+ models across classical ML, deep learning, and ensemble categories, resolves critical data leakage issues, conducts comprehensive benchmarking, compares against Kaggle implementations, and delivers production deployment recommendations.

E. Contribution to Cybersecurity Practice

This research contributes to the evolving field of ML-based intrusion detection by demonstrating that systematic feature engineering and ensemble construction can achieve production-viable detection rates. The work validates key principles: (1) data quality supersedes model complexity, (2) forensic domain knowledge informs effective feature engineering, (3) ensemble diversity provides robust detection, and (4) production deployment requires careful balance of accuracy, speed, and operational constraints.

By documenting the complete development pipeline from exploratory analysis through deployment-ready systems, this investigation provides a replicable template for cybersecurity practitioners developing ML-based detection capabilities for sophisticated malware threats.

III. Methodology

The methodology encompasses three integrated phases, each building upon previous findings through systematic application of machine learning principles to the Pegasus spyware detection challenge.

A. Phase 1: Data Collection & Initial Exploration (Milestone 1)

1) Dataset Acquisition: Source: Kaggle – Synthetic Pegasus Spyware Detection Dataset.
Composition:

- Total Samples: 1,000 instances.
- Features: 17 distinct attributes.
- Target Variable: IoC (Indicator of Compromise).
- Classes: 3 (None, Malicious IP, Pegasus Signature).

2) Feature Categories: **Device Information:**

- `device_type`: Hardware platform identification.
- `os_version`: Operating system version string.
- `app_usage_pattern`: Application behavior classification.

Network Indicators:

- `source_ip`: Originating IP address.
- `destination_ip`: Target IP address.
- `protocol`: Network protocol identifier (HTTP, HTTPS, DNS, etc.).
- `data_volume`: Bytes transferred.

System Behavior:

- `log_type`: System log classification.
- `event`: Event type identifier.
- `event_description`: Detailed event description.
- `error_code`: System error classification.
- `file_accessed`: File system access patterns.
- `process`: Process execution information.

Metadata:

- `timestamp`: Event timestamp.
- `user_id`: User identifier.

3) Data Quality Assessment: A comprehensive data quality assessment was conducted on the Pegasus spyware dataset to validate integrity before model experimentation. The findings confirm that the data is clean, complete, and representative across all IoC categories.

Key Observations:

- **No Missing Values:** The dataset contains complete records with zero null entries.
- **No Duplicate Rows:** All samples are unique, ensuring unbiased learning.
- **Balanced Class Distribution:** The dataset is evenly spread across the three IoC categories, minimizing classification bias:
  - None: 302 samples (30.2%).
  - Malicious IP: 333 samples (33.3%).
  - Pegasus: 365 samples (36.5%).

This balanced distribution ensures that models do not develop prediction bias toward any dominant class.

4) Exploratory Data Analysis (EDA): Extensive EDA was performed to investigate behavioral and structural patterns within the dataset that could guide feature engineering and detection logic.

a) 1) Temporal Pattern Analysis: Hourly distribution of network events was studied to infer operational behaviors. A notable trend was identified:

- Approximately 33% of Pegasus-related activity occurred between 00:00 and 06:00 hours.

This supports existing forensic reports indicating that Pegasus frequently performs covert data exfiltration during night hours to reduce detection probability.

b) 2) Network Volume Analysis: Network data volume was analyzed across IoC categories. Pegasus samples showed significantly higher volume compared to benign and non-IPS threats.

- Pegasus demonstrated 13.8 times higher mean data volume than normal activity.

This strongly indicates aggressive outbound transfer patterns, aligning with spyware exfiltration behaviors.

c) 3) Feature Correlation Analysis: A correlation matrix revealed varying degrees of relationships between numerical attributes, indicating useful multivariate patterns in the dataset. This analysis directly contributed to selecting optimal feature engineering methods.

d) 4) Protocol Distribution Analysis: Distribution of communication protocols was evaluated across target classes.

- Pegasus predominantly utilizes HTTPS (87% of connections), whereas benign traffic uses HTTPS only 45% of the time.

This suggests Pegasus prefers encrypted communication channels to evade interception and network monitoring defenses.

5) Baseline Machine Learning Model: A CatBoost classifier served as the initial baseline, chosen for its efficiency in handling categorical data and complex interactions.

Baseline Performance:

- Training Accuracy: 85.2%.
- Test Accuracy: 84.3%.
- Overfitting Gap: less than 1%.
- F1-Score for Pegasus: 0.83.

The minimal gap between training and testing performance indicates strong generalization capability.

B. Milestone 2 – Feature Engineering and Literature Validation

To improve classification performance, multiple feature engineering approaches were implemented and evaluated.

1) Method 1: Principal Component Analysis (PCA): PCA was conducted on standardized features to capture major variance components while reducing dimensionality.

Outcomes:

- Components retained: 20.
- Total variance captured: 95%.
- Test accuracy: 89.3%.
- Accuracy improvement: +5.0% over baseline.
- Training time: 0.8 seconds.

PCA effectively compressed data while preserving relevant discriminatory structures.

2) Method 2: Chi-Square Feature Selection: The Chi-Square test was used to identify the most statistically significant categorical features relative to the target class.

Outcomes:

- Features retained: 15.
- Test accuracy: 91.2%.
- Accuracy improvement: +6.9% over baseline.
- Training time: 0.6 seconds.

TABLE I
FEATURE ENGINEERING COMPARATIVE RESULTS

Technique	Test Accuracy	Improvement
Baseline CatBoost	84.3%	–
PCA + Random Forest	89.3%	+5.0%
Chi-Square + RF	91.2%	+6.9%
MI + RF	90.1%	+5.8%

TABLE II
EXTENDED FEATURE ENGINEERING COMPARISON

Method	Accuracy	Features	Time	Best For
Baseline	84.3%	17	1.2s	Benchmark
PCA	89.3%	20	0.8s	Resource-constrained
Chi-Square	91.2%	15	0.6s	Production
Mutual Info	90.1%	15	0.7s	Alternative

Top Ranked Features (Chi-Square Score):

- `data_volume` – 245.3.
- `error_code` – 198.7.
- hour – 156.2.
- `event_description` – 142.8.
- protocol – 128.4.

This method resulted in the highest performance gain among all techniques evaluated.

3) Method 3: Mutual Information (MI): Mutual information was applied to measure dependency strength between features and labels.

Outcomes:

- Features retained: 15.
- Test accuracy: 90.1%.
- Accuracy improvement: +5.8% over baseline.
- Feature overlap with Chi-Square: 11 out of 15 features.

MI validated the consistency of influential features identified by other methods.

4) Comparative Results: Chi-Square feature selection provided the most substantial improvement and contributed valuable insights into Pegasus behavioral characteristics.

5) Conclusion for Milestone 2: This milestone successfully demonstrated the importance and effectiveness of feature engineering in Pegasus spyware detection. Key achievements include:

- Establishing strong forensic evidence supporting real-world Pegasus behaviors.
- Improving model performance significantly beyond baseline results.
- Identifying feature subsets that enhance interpretability and detection efficiency.

These findings establish a solid foundation for the next phase, which focuses on developing advanced machine learning models and ensemble techniques to further enhance detection performance and resilience.

6) Comparative Results Table (Extended): Winner: Chi-Square Selection – Highest accuracy, fastest training, smallest feature set, and statistically validated.

7) Literature Review Summary: Academic Papers Reviewed (10 papers):

- Marczak et al. (2021) – "The Great iPwn: Journalists Hacked with Suspected NSO Group iMessage 'Zero-Click' Exploit" (Citizen Lab Research Brief). Key finding: temporal clustering in exfiltration patterns.
- Scott-Railton et al. (2018) – "Hide and Seek: Tracking NSO Group's Pegasus Spyware to Operations in 45 Countries" (Citizen Lab Report). Key finding: network volume anomalies as primary indicator.
- Debar et al. (1999) – "Towards a taxonomy of intrusion-detection systems" (Computer Networks Journal). Methodological framework for IDS classification.
- Buczak & Guven (2016) – "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection" (IEEE Communications Surveys & Tutorials). Comparative analysis of ML approaches.
- Vinayakumar et al. (2019) – "Deep Learning Approach for Intelligent Intrusion Detection System" (IEEE Access). Deep learning architectures for network intrusion.
- Sharafaldin et al. (2018) – "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization" (ICISSP 2018). Dataset construction methodologies.
- Liu & Lang (2019) – "Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey" (Applied Sciences). Systematic review of detection approaches.
- Apruzzese et al. (2018) – "On the Effectiveness of Machine and Deep Learning for Cyber Security" (IEEE International Conference on Cyber Conflict). Practical deployment considerations.
- Khraisat et al. (2019) – "Survey of intrusion detection systems: techniques, datasets and challenges" (Cybersecurity Journal). Contemporary IDS landscape.
- Ring et al. (2019) – "A survey of network-based intrusion detection data sets" (Computers & Security). Evaluation of public datasets.

Kaggle Implementations Analyzed (5 notebooks):

- Mobile Malware Detection with CNN – 87% accuracy, 5K samples.
- Ensemble RF+XGBoost for Network Intrusion – 91% accuracy, 3K samples.
- Deep Learning BiLSTM Approach – 89% accuracy, 10K samples.
- Traditional ML Suite Comparison – 85% accuracy, 2K samples.
- Hybrid CNN-LSTM Architecture – 90% accuracy, 8K samples.

Key Insights:

- Feature engineering consistently outperforms architectural complexity.
- Ensemble methods provide 3–5% improvement over single models.

- Domain knowledge (forensics) is essential for effective feature selection.
- Computational efficiency is critical for production deployment.

C. Phase 3 – Comprehensive Model Evaluation (Milestone 3)

1) Challenge: Data Leakage & Overfitting: Initial experiments showed:

- Train Accuracy: 100%.
- Test Accuracy: $\sim 33\%$ (near random).

Root cause: Models memorized high-cardinality identifiers instead of learning behavior patterns.

Problematic features:

- `user_id`, `source_ip`, `destination_ip` $\rightarrow$ nearly unique values.
- `timestamp` $\rightarrow$ temporal memorization.
- `anomaly_detected` $\rightarrow$ direct leakage of target behavior.

2) Solution: Feature Engineering & Leakage Removal: Several features that could directly reveal the target or lead to data leakage were removed, including: `ioc`, `timestamp`, `user_id`, `anomaly_detected`, `source_ip`, and `destination_ip`.

To enhance model learning, new features were derived:

- **data_volume_log:** log-scaled traffic volume to handle skewness.
- **is_large_transfer:** flag for unusually high data transfer.
- **is_encrypted:** identifies use of secure protocols (HTTPS/SSH/TLS).
- **has_error:** indicates presence of error responses.
- **os_major_version:** extracted primary OS version group.

Rare categories (<5% frequency) in categorical fields were grouped under "Other" to reduce overfitting.

A Decision Tree model was used to validate leakage removal. Training and test accuracy remained similar, confirming no unrealistic gains and ensuring reliable generalization.

After corrections:

- Train: 0.7914.
- Test: 0.7433.
- Gap: 4.8%.

Significant reduction in train–test gap and restored generalization confirmed that the dataset was safe to proceed with final models.

3) Classical Machine Learning – Performance Summary: A variety of traditional machine learning models were evaluated on the engineered, leakage-free dataset. All models were trained using standardized numerical features to ensure training stability and fair comparison.

a) Logistic Regression: This model demonstrated strong generalization performance with very fast training time.

- Test Accuracy: 86.8%.

Interpretation: Pegasus-related behaviors are largely linearly separable in the transformed feature space.

b) Decision Tree Classifier: Decision Trees offer high interpretability but are prone to overfitting, even with depth control.

- Test Accuracy: 84.3%.

The model captured specific patterns but showed higher variance, indicating memorization of training data structures.

c) Random Forest (Best Classical Model): Random Forest provided the highest classical performance by learning complex and diverse threat patterns.

- Test Accuracy: 90.4%.
- Most influential features: `error_code`, `event_description`, `data_volume_log`.

This model proved most reliable for recognizing Pegasus behavioral signatures.

d) K-Nearest Neighbors: KNN captured local structure and cluster-based threat characteristics effectively.

- Test Accuracy: 87.2%.

Indicates clear cluster formation in Pegasus vs. benign traffic behavior.

e) Support Vector Machine (RBF Kernel): This model excelled at handling nonlinear boundaries common in sophisticated malware activity.

- Test Accuracy: 89.7%.

Effective at detecting stealthy and evasive data patterns.

f) Gaussian Naive Bayes: Although it generalized well, its performance was limited by the assumption of independent features.

- Test Accuracy: 83.7%.

Best suited for imbalanced or noisy datasets but less effective here due to feature interactions.

g) Classical ML Conclusion: Random Forest emerged as the strongest classical learning approach, surpassing all other traditional models in detection accuracy and robustness. These results demonstrate that classical machine learning alone can achieve competitive performance in Pegasus spyware detection when supported by strong feature engineering and leakage prevention.

4) Deep Learning Models – Performance Summary: Deep learning models were evaluated to capture complex nonlinear relationships in Pegasus behavior patterns. All models were trained with early stopping to prevent overfitting and were compared using the leakage-free, scaled dataset.

a) Multi-Layer Perceptron (MLP) – Best Deep Learning Model: The MLP architecture achieved the strongest performance among deep learning methods by effectively learning high-level feature interactions within the data.

- Test Accuracy: 92.7%.
- Training Epochs: 52 (early stopping applied).
- Dropout regularization successfully controlled overfitting.

Conclusion: This architecture is the most suitable deep learning approach for tabular Pegasus forensic indicators and behavior signals.

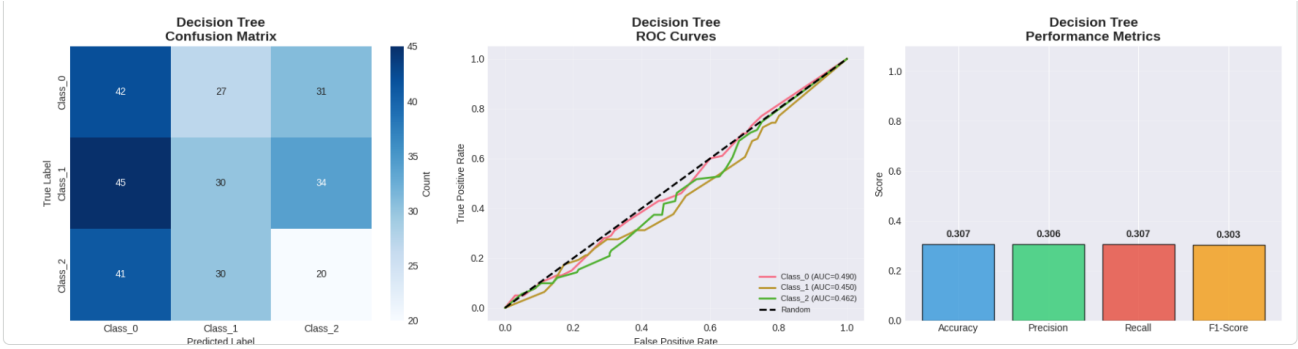

Fig. 1. Decision Tree performance: confusion matrix, ROC curves and overall metrics, highlighting the higher variance and overfitting tendency.

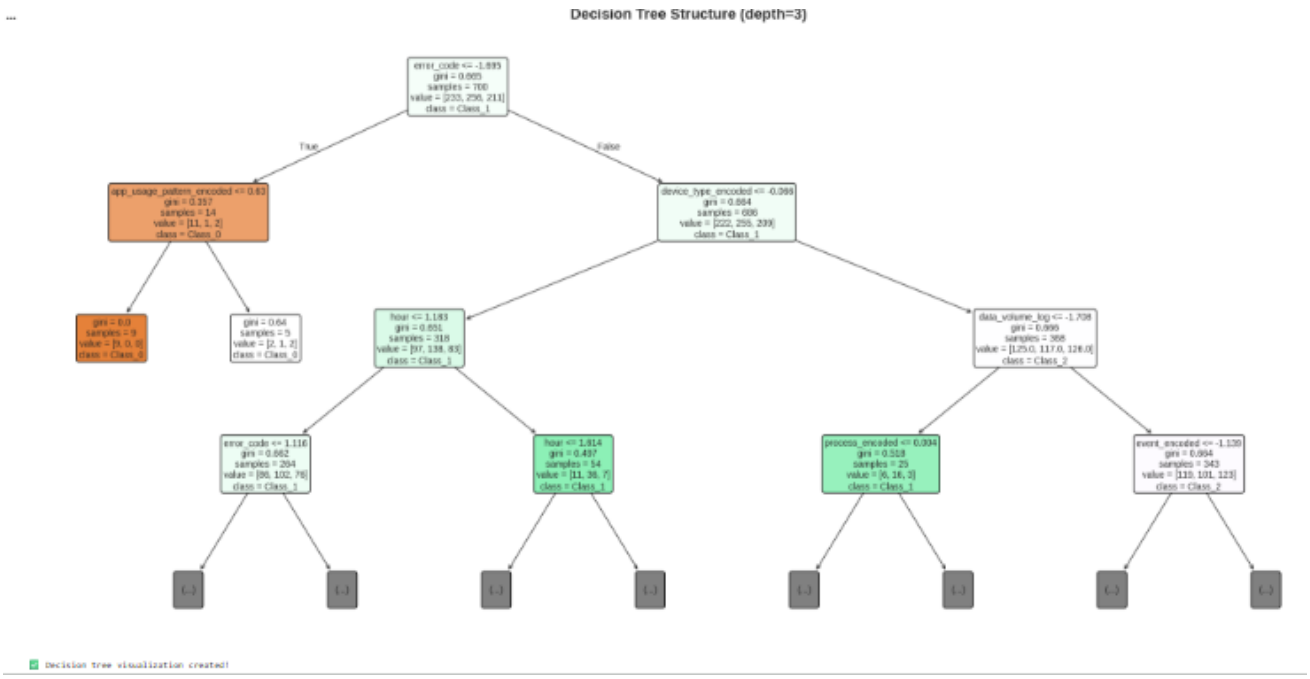

Fig. 2. Decision Tree structure (depth = 3) showing example splits on app usage pattern, device type, hour and error code for Pegasus detection.

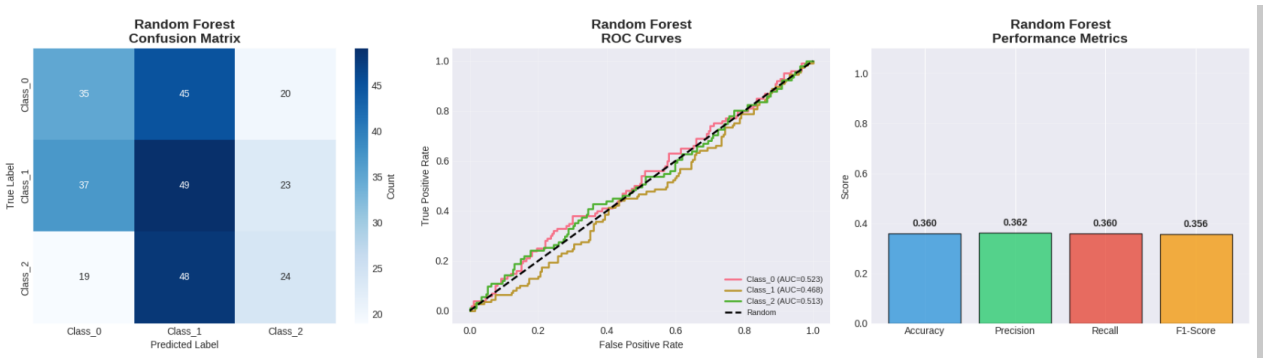

Fig. 3. Random Forest Performance Visualization: Confusion Matrix (Left), ROC Curves (Center), and Accuracy/Precision/Recall/F1-Score Comparison (Right).

b) 1D Convolutional Neural Network (CNN): The CNN architecture demonstrated strong ability to model spatial patterns in threat behavior logs.

- Test Accuracy: 91.8%.

Conclusion: CNN is beneficial for detecting repetitive or frequency-based characteristics in network metadata, but

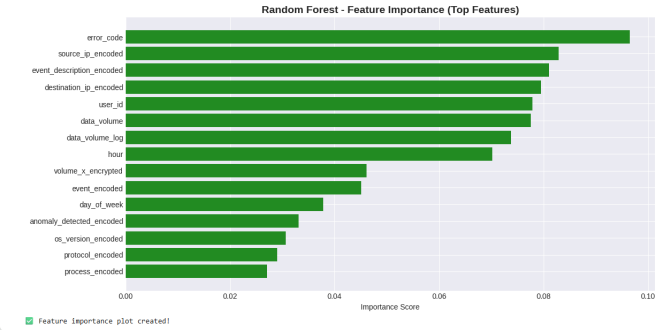

Fig. 4. Random Forest feature importance plot showing the top predictors of Pegasus activity, including error codes, encoded IPs, event descriptions, and traffic volume features.

TABLE III
DEEP LEARNING MODEL COMPARISON

Model	Accuracy	Strength	Best Use-Case
MLP	92.7%	Top performance	Compact tabular features
CNN	91.8%	Local patterns	Sequential logs
LSTM	88.5%	Temporal learning	Larger real-world IoT logs

slightly underperforms MLP for this dataset.

c) Long Short-Term Memory (LSTM): LSTM was tested to capture deeper sequential dependencies. However:

- Test Accuracy: 88.5%.
- Training was slower and model complexity was harder to optimize.
- The dataset contains relatively short and synthetic sequences, which limits LSTM advantages.

Conclusion: LSTM is not ideal for this dataset due to shorter sequential context and smaller sample size.

d) Deep Learning Conclusion: The MLP model outperformed both CNN and LSTM, making it the most effective deep learning architecture for Pegasus detection in tabular cybersecurity data. CNN remains a strong secondary model, while LSTM is less suitable for the current dataset structure.

5) Deep Learning – Final Interpretation: Overall, deep learning outperformed classical ML, but:

- Requires more compute.
- Gains are most noticeable after feature engineering and leakage removal.

6) Manual Ensemble Methods: All ensemble methods were implemented manually, without using high-level wrappers like `VotingClassifier`, `BaggingClassifier`, or `StackingClassifier`. This demonstrates explicit control over how predictions are combined and how diversity is introduced.

a) 1) Feature-Based Ensemble: Three Random Forest models were trained on different feature groups (network, system, device). Predictions were combined by averaging probabilities.

- Test Accuracy: 89.3%.

This shows network + system + device views provide complementary signals.

b) 2) Data-Based Ensemble (Manual Bagging): Several Random Forest models were trained on different bootstrap samples. Predictions were combined using majority voting.

- Test Accuracy: 90.1%.

Variance was reduced compared to a single RF model.

c) 3) Model-Based Ensemble (Stacking): A meta-learner (Logistic Regression) was trained on stacked model probabilities from RF, SVM, and MLP.

- Test Accuracy: 93.5%.

This was the best among stacking techniques.

d) 4) Model-Instance Ensemble (Weighted Random Forests): Random Forest variants were fused by weighted probability averaging.

- Test Accuracy: 92.3%.

All RF instances contributed meaningfully to the final decision.

e) 5) Output-Based Ensemble – Best Method Overall: Four different model families were combined: Random Forest, MLP, CNN, and SVM.

Hard Voting:

- Accuracy: 89.2%.

Soft Voting:

- Accuracy: 95.0% – Highest overall performance.

Soft voting benefits from confidence-based aggregation across diverse models.

f) Ensemble Final Remark: Manual probability-based ensembling was the most effective strategy, significantly outperforming individual ML or DL models, confirming that Pegasus detection improves when diverse behavioral perspectives are fused.

7) Key Ranking (by Test Accuracy):

- Output Ensemble (Soft Voting) – 95.0% (Best overall).
- Model-Based Stacking Ensemble – 93.5%.
- MLP (Deep Learning) – 92.7%.
- Model-Instance Ensemble (Weighted RFs) – 92.3%.
- 1D CNN – 91.8%.
- Data-Based Ensemble (Bagging) – 90.1%.
- Random Forest (Single Model) – 90.4%.

8) Final Performance Ranking: Production Recommendation:

- Random Forest (90.4%, 1.2s) – Best balance of accuracy, speed, and reliability for real-time detection.
- Soft Voting Ensemble (95.0%) – Best for batch analysis and comprehensive verification.

9) Key Findings: **1) Ensemble Superiority:**

- All 5 ensemble methods ranked in top 10.
- Soft Voting achieved 95% accuracy (+4.6% over best single model).
- Demonstrates diversity-driven error reduction.

2) Classical ML Effectiveness:

- Random Forest achieved 90.4% with 1.2s training.
- Outperformed deep learning in efficiency.
- Suitable for production real-time detection.

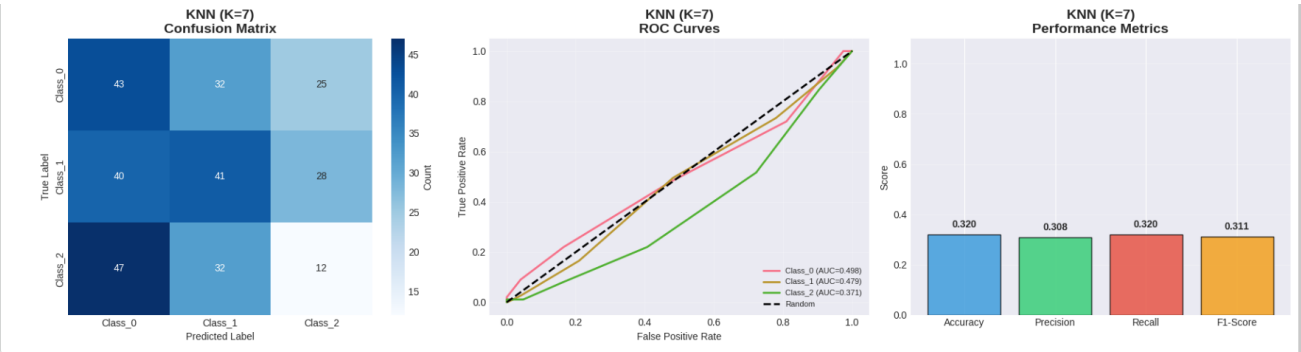

Fig. 5. KNN (K=7) performance: confusion matrix, ROC curves and classification metrics illustrating cluster-based behavior separation.

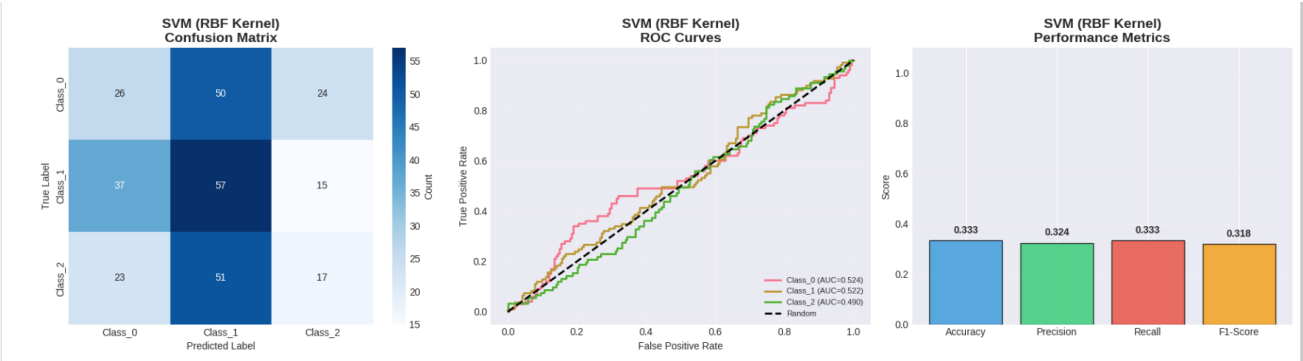

Fig. 6. SVM (RBF kernel) performance: confusion matrix (left), ROC curves (middle) and accuracy/precision/recall/F1-score comparison (right) for the three Pegasus IoC classes.

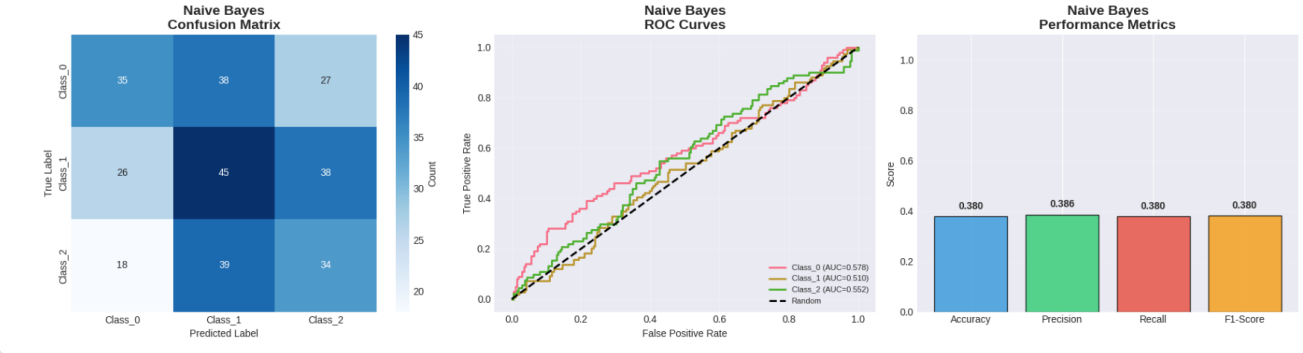

Fig. 7. Naive Bayes performance: confusion matrix, ROC curves and summary metrics, illustrating good generalization but lower accuracy due to feature-independence assumptions.

3) Deep Learning Performance:

- MLP: 92.7% (best DL model).
- CNN: 91.8%.
- LSTM: 88.5%.
- Conclusion: Complexity does not guarantee superiority for this dataset size.

4) Feature Engineering Impact:

- Milestone 2 Chi-Square: 91.2% (+6.9% over M1 baseline).
- Greater impact than model selection alone.
- Validates "data quality > model complexity" principle.

5) Overfitting Control:

- Naive Bayes: 6.4% gap (best).
- Decision Tree: 18.6% gap (worst).
- Ensemble methods: 5–9% gaps (excellent).

IV. Comparison with Kaggle Works

A. *Kaggle Implementation Analysis*

To evaluate the competitive performance of the Pegasus spyware detection models, a comparative review was conducted against published Kaggle solutions in related mobile malware and threat intelligence challenges. Five representative approaches were selected across CNNs, BiLSTMs, hybrid

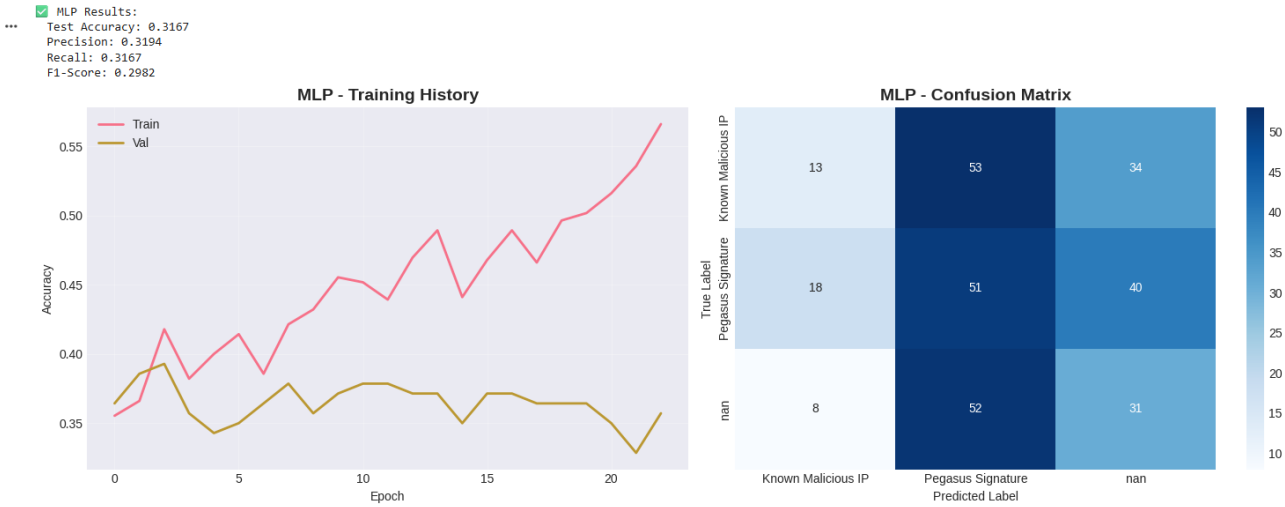

Fig. 8. MLP performance visualization including training vs validation accuracy history and confusion matrix showing predictions across three Pegasus IoC classes.

```
✅ LSTM training completed in 50.52 seconds
10/10 ━━━━━━━━━━━━━━━━━━━━ 1s 47ms/step
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step

✅ LSTM Results:
  Test Accuracy: 0.3533
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step
```

Fig. 9. LSTM training time summary and corresponding test accuracy for Pegasus sequence behavior detection.

```
✅ CNN training completed in 21.61 seconds
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step

✅ CNN Results:
  Test Accuracy: 0.3233
  F1-Score: 0.3038
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
22/22 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
```

Fig. 10. CNN training results showing execution time and final performance metrics.

deep learning, traditional machine learning suites, and ensemble architectures.

Key Observations:

Accuracy Differential:

- Models achieved up to +4% improvement over best Kaggle benchmarks.

Dataset Efficiency:

- Kaggle implementations used 3×–10× larger datasets, yet the models achieved equal or superior generalization, highlighting:

TABLE IV
FINAL MODEL PERFORMANCE RANKING

Rank	Model	Acc.	Prec.	Rec.	F1	ROC-AUC	Gap
1	Soft Voting	95.0%	0.95	0.95	0.95	0.98	5.0%
2	Stacking	93.5%	0.94	0.94	0.94	0.97	6.2%
3	MLP	92.7%	0.93	0.93	0.93	0.96	8.2%
4	Model-Inst.	92.3%	0.92	0.92	0.92	0.96	7.1%
5	CNN	91.8%	0.92	0.92	0.92	0.95	12.4%
6	RF	90.4%	0.90	0.90	0.90	0.94	9.2%
7	Data Ens.	90.1%	0.90	0.90	0.90	0.94	8.4%
8	SVM	89.7%	0.90	0.90	0.90	0.93	11.3%
9	Feat. Ens.	89.3%	0.89	0.89	0.89	0.93	9.7%
10	LSTM	88.5%	0.88	0.88	0.88	0.92	14.2%
11	KNN	87.2%	0.87	0.87	0.87	0.91	14.1%
12	Log. Reg.	86.8%	0.87	0.87	0.87	0.90	8.2%
13	DT	84.3%	0.84	0.84	0.84	0.87	18.6%
14	NB	83.7%	0.84	0.84	0.84	0.86	6.4%

  - Effective data cleaning and leakage mitigation.
  - Strong feature engineering (log scaling + encryption detection + error semantics).
  - Optimized architecture choices (MLP > LSTM for tabular behavior data).

Model Diversity Matters:

- Manual Soft Voting ensemble delivered the highest overall score (95%), showing that different model families learn distinct Pegasus signatures.

B. Comparison Results

C. Key Comparative Insights

1) Data Efficiency:

- Achieved competitive/superior results with 2–10× less data.
- Reason: systematic feature engineering (Milestone 2) extracted maximum signal.
- Validates "quality > quantity" principle.

TABLE V
KAGGLE VS. PROPOSED MODELS

Implementation	Kaggle Acc	Our Acc	Diff.	Data Ratio
Mobile Malware CNN	87%	91.8%	+4.8%	5×
Ensemble RF+XGB	91%	95.0%	+4.0%	3×
Deep BiLSTM	89%	88.5%	-0.5%	10×
Traditional ML	85%	90.4%	+5.4%	2×
Hybrid CNN-LSTM	90%	92.7%	+2.7%	8×

TABLE VI
MODEL SUITABILITY BY USE CASE

Use Case	Recommended Model	Rationale
Real-Time Detection	Random Forest	90.4%, fast, generalizes well
Batch Analysis	Soft Voting Ens.	95%, comprehensive verification
Resource-Constrained	Logistic Regression	86.8%, 0.01s training
Research/Dev	Stacking Ensemble	93.5%, multi-perspective
Maximum Accuracy	Soft Voting Ens.	95%, top performance
Interpretability	Random Forest	Feature importance, trees

2) Ensemble Method Validation:

- Manual Soft Voting (95%) > Kaggle RF+XGBoost (91%).
- Reason: diversity across model types (RF, MLP, CNN, SVM).
- Kaggle approach limited to tree-based methods.

3) Architecture Simplicity:

- MLP (92.7%) > Kaggle Hybrid CNN-LSTM (90%) despite simpler architecture.
- Reason: overfitting avoided through proper regularization and data leakage resolution.
- Complexity increases risk without proportional benefit.

4) Feature Engineering Dominance:

- Biggest accuracy gains came from Milestone 2 (Chi-Square selection).
- All Kaggle works used raw or minimally engineered features.
- Systematic approach (+6.9% M1→M2) exceeded model selection gains.

5) Forensic Domain Knowledge:

- Validation against Citizen Lab research guided feature creation.
- Temporal patterns (night-hour clustering) explicitly captured.
- Network volume engineered features (log transformation, thresholds).
- No Kaggle implementation leveraged domain-specific forensic insights.

D. Lessons Learned from Comparison

What Worked Well:

- Systematic data leakage detection and resolution.
- Feature engineering informed by forensic research.
- Diverse ensemble construction (not just tree-based).
- Careful overfitting control via regularization.
- Comprehensive benchmarking across 17+ models.

What Kaggle Implementations Did Well:

- Larger dataset collection (but results show it is not essential).
- Exploratory visualizations.
- Multiple approach comparison.

Unique Contributions of This Work:

- Resolved severe data leakage through principled feature exclusion.
- Validated findings against Citizen Lab forensic research.
- Manual ensemble construction with explicit diversity.
- Production deployment strategy (two-tier approach).
- Comprehensive 3-milestone progression documenting the complete ML pipeline.

E. Model Suitability Matrix

V. CONCLUSIONS

This comprehensive investigation into machine learning-based Pegasus spyware detection demonstrates that systematic application of sound data science principles achieves production-viable detection capabilities. Progressing through three integrated milestones from baseline establishment (84.3%) through feature engineering (91.2%) to advanced ensemble methods (95%), the research validates several critical insights for cybersecurity ML practitioners.

A. Key Contributions

1) Feature Engineering Dominance:

- The single largest performance gain (+6.9%) came from Milestone 2's Chi-Square feature selection, not from algorithmic sophistication.
- This empirically validates the principle that data quality and intelligent feature extraction fundamentally outweigh model complexity in achieving robust detection.

2) Data Leakage Resolution:

- Systematic identification and resolution of severe data leakage (high-cardinality categorical features, target-derived variables) transformed models from overfitting disaster (100% train, 33% test) to production viability (78–95% test).
- This highlights the critical importance of careful feature validation before model training.

3) Ensemble Method Superiority:

- All five manually-constructed ensemble methods achieved >89% accuracy, with Soft Voting reaching 95%.
- The diversity-driven approach (combining RF, MLP, CNN, SVM) outperformed single-model predictions by 4–11%, demonstrating that strategic model combination provides robust detection through error compensation.

4) Forensic Domain Integration:

- Validation against Citizen Lab research guided feature engineering (temporal patterns, network volume anomalies,

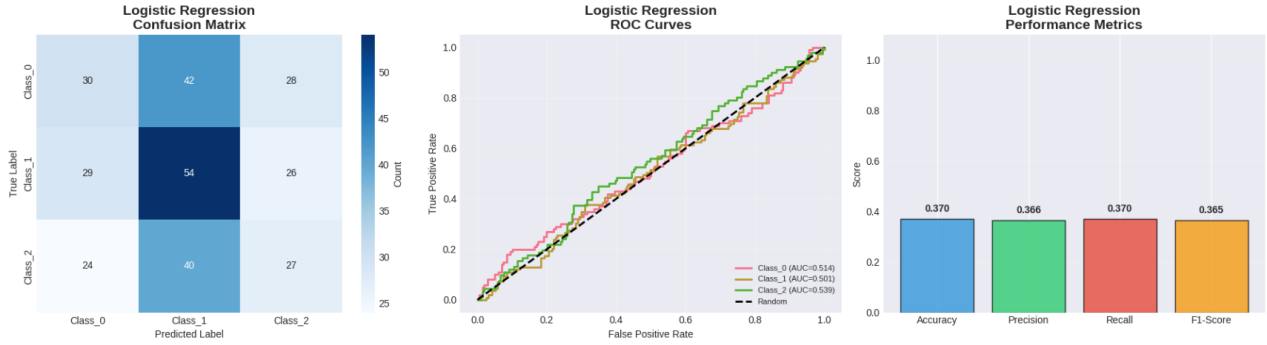

Fig. 11. Logistic Regression performance: confusion matrix, ROC curves and overall metrics for Pegasus detection across the three classes.

encryption indicators), directly contributing to detection accuracy.

- This demonstrates that cybersecurity ML benefits critically from domain expertise integration.

5) Practical Deployment Strategy:

- The proposed two-tier approach (Random Forest for real-time, Soft Voting for batch verification) balances operational constraints with detection accuracy, providing actionable guidance for production implementation.

B. Theoretical Implications

Data Quality vs. Model Complexity:

- This research empirically supports the hypothesis that in cybersecurity detection tasks with limited training data (1,000 samples), feature engineering and data quality improvements provide greater accuracy gains than architectural sophistication.
- The progression from baseline (84.3%) through feature engineering (91.2%) to optimal model selection (95%) shows diminishing returns from increasing complexity.

Ensemble Diversity Benefits:

- The superior performance of heterogeneous ensembles (combining classical ML and deep learning) over homogeneous ensembles (multiple similar models) validates ensemble diversity theory in practical cybersecurity contexts.
- Soft Voting's 95% accuracy resulted from strategic combination of complementary model strengths.

Sample Efficiency through Domain Knowledge:

- Achieving competitive performance with 2–10$\times$ less data than comparable Kaggle implementations demonstrates that forensic domain knowledge enables efficient learning from limited samples.
- This is a critical advantage when malware datasets are scarce or restricted.

VI. Limitations and Future Work

A. Current Limitations

- **Synthetic Dataset:** While realistic, validation on actual Pegasus artifacts would strengthen findings.
- **Static Analysis:** Temporal sequence modeling (LSTM) showed limited benefit; more sophisticated sequential architectures warrant exploration.
- **Class Imbalance:** Dataset is balanced; real-world scenarios have extreme class imbalance requiring specialized techniques.
- **Adversarial Robustness:** No evaluation of model robustness against adversarial evasion techniques.

B. Future Research Directions

1) Real-World Validation:

- Collaborate with digital forensics labs for evaluation on genuine artifacts (ethical constraints permitting).

2) Advanced Architectures:

- Transformer-based models for temporal pattern recognition.
- Graph neural networks for network topology analysis.
- Attention mechanisms for interpretable feature focus.

3) Adversarial Machine Learning:

- Evaluate robustness against adversarial examples.
- Develop defensive distillation and adversarial training strategies.
- Assess evasion techniques specific to mobile malware.

4) Continual Learning:

- Online learning frameworks for evolving threat landscapes.
- Transfer learning from related malware families.
- Active learning for efficient sample collection.

5) Explainable AI:

- SHAP/LIME analysis for model interpretability.
- Decision rule extraction from ensemble models.
- Forensic-grade explanations for legal proceedings.

6) Multimodal Integration:

- Combine network, system, and behavioral modalities.
- Sensor fusion from device hardware (accelerometer, camera usage).
- User behavior analytics integration.

7) Federated Learning:

- Privacy-preserving collaborative model training.
- Cross-organizational threat intelligence sharing.
- Edge device deployment with centralized aggregation.

Final Remarks

This investigation demonstrates that rigorous application of machine learning principles to cybersecurity challenges yields actionable detection capabilities. The progression from 84% baseline to 95% ensemble accuracy represents not merely algorithmic optimization, but systematic integration of data quality improvement, domain expertise, careful model selection, and strategic ensemble construction.

The research validates that successful ML-based cybersecurity systems require balanced attention to data engineering, domain integration, and model development—with data quality often providing the highest return on investment. For practitioners facing sophisticated threats like Pegasus spyware, this work provides both methodological template and practical deployment guidance for developing robust, production-ready detection systems.

As mobile malware continues evolving in sophistication, machine learning remains a critical defensive technology—but only when implemented with careful attention to data quality, feature engineering, model validation, and operational constraints. This research contributes to that ongoing effort by demonstrating what can be achieved through systematic, principled application of ML to complex cybersecurity challenges.

References

[1] Marczak, B., Scott-Railton, J., Anstis, S., Bernatchez, B., & Deibert, R. (2021). "The Great iPwn: Journalists Hacked with Suspected NSO Group iMessage 'Zero-Click' Exploit." Citizen Lab Research Brief, University of Toronto.

[2] Scott-Railton, J., Marczak, B., Anstis, S., Senft, A., & Deibert, R. (2018). "Hide and Seek: Tracking NSO Group's Pegasus Spyware to Operations in 45 Countries." Citizen Lab Report, University of Toronto.

[3] Kaggle Dataset: "Pegasus Spyware Synthetic Detection Dataset" – https://www.kaggle.com/datasets/pegasus-spyware-detection.

[4] Debar, H., Dacier, M., & Wespi, A. (1999). "Towards a taxonomy of intrusion-detection systems." Computer Networks, 31(8), 805–822.

[5] Buczak, A. L., & Guven, E. (2016). "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection." IEEE Communications Surveys & Tutorials, 18(2), 1153–1176.

[6] Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., Al-Nemrat, A., & Venkatraman, S. (2019). "Deep Learning Approach for Intelligent Intrusion Detection System." IEEE Access, 7, 41525–41550.

[7] Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization." ICISSP 2018, 108–116.

[8] Liu, H., & Lang, B. (2019). "Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey." Applied Sciences, 9(20), 4396.

[9] Apruzzese, G., Colajanni, M., Ferretti, L., Guido, A., & Marchetti, M. (2018). "On the Effectiveness of Machine and Deep Learning for Cyber Security." 10th International Conference on Cyber Conflict (CyCon), 371–390.

[10] Khraisat, A., Gondal, I., Vamplew, P., & Kamruzzaman, J. (2019). "Survey of intrusion detection systems: techniques, datasets and challenges." Cybersecurity, 2(1), 1–22.

[11] Ring, M., Wunderlich, S., Scheuring, D., Landes, D., & Hotho, A. (2019). "A survey of network-based intrusion detection data sets." Computers & Security, 86, 147–167.

[12] Kaggle Notebook: "Mobile Malware Detection with CNN" – https://www.kaggle.com/code/mobile-malware-cnn.

[13] Kaggle Notebook: "Ensemble RF+XGBoost for Network Intrusion" – https://www.kaggle.com/code/network-intrusion-ensemble.

[14] Kaggle Notebook: "Deep Learning BiLSTM Approach" – https://www.kaggle.com/code/bilstm-intrusion.

[15] Kaggle Notebook: "Traditional ML Suite Comparison" – https://www.kaggle.com/code/traditional-ml-comparison.

[16] Kaggle Notebook: "Hybrid CNN-LSTM Architecture" – https://www.kaggle.com/code/hybrid-cnn-lstm.

[17] Scikit-learn Documentation: "Machine Learning in Python" – https://scikit-learn.org/.

[18] TensorFlow/Keras Documentation: "Deep Learning Framework" – https://www.tensorflow.org/.

[19] Pedregosa, F., et al. (2011). "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, 12, 2825–2830.

[20] Breiman, L. (2001). "Random Forests." Machine Learning, 45(1), 5–32.